

Exercise

3\.

Part of the Task Manager	What it does	Example
Task	Stores the information about a task	"Finish homework"
TaskPriority	Determines how important the task is	LOW, MEDIUM, HIGH, URGENT
TaskStatus	Shows the current stage of the task	TODO - IN_PROGRESS - REVIEW - DONE
Task.update()	Changes information about an existing task	Changes the title or priority
Task.markAsDone()	Completes the task	Changes status to DONE
createdAt	Records when the task was created	Date/time of creation
updatedAt	Records when the task was last changed	Date/time of latest update
storage	Keeps the task data saved	Saves and retrieves tasks

4\.

Insights:	What i learned:
Task model	The task class stores important information about a task, such as its title, description, priority, status, dates and tags.
Task priority	Tasks can have different priority levels: LOW, MEDIUM, HIGH and URGENT.
Task status	A task moves through different stages, such as TODO, IN_PROGRESS, REVIEW, and DONE.
Updating tasks	The update() function changes existing task information and records when the task was updated.
Completing tasks	The markAsDone() function changes the task's status to DONE.
Code organisation	Different files have different responsibilities, which makes the application easier to understand and maintain.
Tracing code	I learned that i can follow where data comes from, which function changes it, and where the updated information goes.
Using AI explanations	AI helped me understand unfamiliar code by breaking it into smaller sections and explaining the purpose of each part.

Reflection Questions:

1. The AI's explanation helped me understand the algorithm more clearly. Before, I found the conditions and loops confusing and was not sure how the different parts connected. The explanation showed me the steps in order and helped me understand how the algorithm makes decisions and processes the task. I now have a better understanding of how the different parts of the code work together.

2\.. After the AI explanation, I still found some of the nested conditions and loops difficult to understand. It was sometimes hard to follow which condition would run first and how the loop affected the next steps. I also needed to look at the code several times to understand how the different parts connected.

3\.. I would explain the algorithm by breaking it into small steps. First, I would explain what the algorithm is trying to achieve. Then I would go through the conditions and loops one at a time and explain what happens when each condition is true or false. I would also use a simple example to show how the data moves through the algorithm. This would make it easier for a junior developer to understand the logic without trying to understand the whole algorithm at once.

4\.. Yes, I tested my understanding against the AI by explaining how I thought the algorithm worked and comparing it with the AI's explanation. This helped me identify parts that I misunderstood and confirm the parts I understood correctly. It also helped me understand the conditions, loops, and overall flow of the algorithm better.

5\.. Based on my understanding, I would improve the algorithm by making the logic simpler and easier to follow. I would reduce unnecessary nested conditions and break the larger sections into smaller functions where possible. I would also make the code

easier to read by using clear variable names and adding comments where the logic is difficult to understand. This would make the algorithm easier to maintain and debug.

Algorithm 1 : Task priority sorting algorithm (JavaScript)

* I would explain that the algorithm gives each task a score based on how important or urgent it is. It first gives points based on the task's priority, then adds points if the due date is close or overdue. It reduces the score for tasks that are completed or in review. It can also add points for certain tags and recently updated tasks. After calculating the scores, the tasks are sorted from the highest score to the lowest score, so the most important tasks appear first.

Algorithm 2 : Task Text Parser (JavaScript)

* The AI's explanation helped me understand how the algorithm takes normal text and breaks it down into different task properties. Before, I did not fully understand how the priority, tags, and dates were identified. The explanation helped me see how the algorithm removes these markers from the text and uses them to create a structured task object.

Algorithm 3 : Task list merging (Two - way sync) (JavaScript)

* The algorithm compares task lists from two different sources, local and remote. It checks which tasks exist in each list and combines them into one list. When the same task has different information, it resolves the conflict by comparing the update dates

and using the newer information. It also gives priority to completed tasks and combines tags from both versions. This allows the changes between the two task lists to be synchronized.